AMAURA LABS

Amaura Explained Simply

Stage 1 Research Notes - corrected final learning edition
Through October 2026

Prepared for Akshat Singh
Companion guide to the Stage 1 research paper. Written in simple language with explicit notes about what is proved, what is not, and what still needs testing.

Important: The Experiment 16 result is now classified as reused / contaminated, not as fresh validation.

1. Amaura in one sentence

Amaura asks AI for many possible solutions, tests those solutions automatically, and keeps the ones that actually work best.

Normal AI is mostly: question -> answer. Amaura is: question -> many candidates -> tests -> comparison -> winner -> verification.

Note: Amaura is not a new AI model. The AI model supplies ideas. Amaura supplies the laboratory around those ideas.

2. The Maggi Example

Imagine a cook knows 1,000 Maggi recipes. You ask for one great recipe and the cook gives you one. Maybe it is good. Maybe not.

Amaura says: “Give me 50 different recipes.” Then all 50 are actually cooked and scored. Recipe 37 wins. The important thing is not that Amaura magically knew Recipe 37. The important thing is that it searched through possibilities and measured the result.

Core lesson: AI can be the cook; the evaluator is the taste test; search is trying many recipes.

3. The Real Hypothesis

The original dream was too broad: “AI can discover better solutions.” The corrected Stage 1 hypothesis is narrower:

Core hypothesis: When a problem has a trustworthy way to measure what is good, AI can generate many possible solutions and an automated search system can test them and select a better candidate.

Bigger future hypothesis: A system that keeps structured experience from past evaluated searches may eventually learn how to search better on genuinely new problems without simply replaying old answers. That part is still open.

4. What “Evaluator” Means

Problem	How we can define “better”
Graph coloring	Use fewer colors while remaining valid.
Job scheduling	Lower total completion time / makespan.
Cloud scheduling	Lower infrastructure cost while still running everything.
Code optimization	Same correct behavior, but faster or using less memory.
Website design	Could be higher conversion, retention or trust; this is noisier.
Scientific hypothesis	Could be better agreement with evidence and better performance on new experiments.
Physical design	Simulation or real-world experiment; expensive and slower.

Big idea: The harder the problem, the harder it is to build a trustworthy evaluator. This is one of the biggest things we learned.

5. What Stage 1 Actually Proves

We have evidence for	We do NOT have proof for
AI can generate executable candidate algorithms.	Amaura can solve almost any problem.
Automated testing can compare candidates.	Amaura always beats an expert prompt.
Search can find better candidates on some optimization tasks.	Evolution is always better than independent sampling.
Graph coloring gave a strong held-out result.	The algorithm is fundamentally new science.
Independent Best-of-N is a strong practical search primitive in our tests.	Past experience transfers automatically to new domains.
Failures and evaluator problems can be discovered and corrected.	Customers will pay for the system.

Remember: “Supported” does not mean “universal.” It means the experiments give real evidence for that narrower statement.

6. Every Experiment - Simple Version

Experiment 1 - Cache policy

We first thought Amaura had found a slightly better cache rule. Then we checked the machinery. The paid AI models had not actually run. A local fallback ran instead, and the generator already contained a hand-written frequency/recency formula.

Verdict: invalid. **Most important lesson:** always prove what actually ran.

Experiment 2 - Genuine small AI

This time the real model ran. It produced valid candidates, but they were worse than the simple baseline. Repeating the loop did not fix the problem.

Verdict: negative. **Lesson:** many AI ideas do not automatically mean good ideas.

Experiments 3-5

The current research archive does not contain enough reliable detail to reconstruct these experiments. We leave them blank instead of inventing results.

Experiment 6 - Small graph-coloring test

AI found a small improvement against DSATUR in one setting, but the statistics were weak and the candidate looked like a combination of known tricks.

Experiment 7 - Big graph-coloring test

This was the first strong result. Amaura produced a graph-coloring solver that combined known ideas: DSATUR restarts, RLF, Kempe-chain reduction and a one-swap squeeze. On 160 hidden graphs it beat DSATUR and RLF on the reported benchmark.

Result: 59 wins / 0 losses / 101 ties vs DSATUR; 37 / 0 / 123 vs RLF. **Important:** good benchmark performance is not proof of a new algorithmic principle.

Experiment 8 - Evolution vs independent ideas

We gave the search methods the same budget. The more complicated evolutionary search did worse than simple independent sampling in the clean PFSP test.

Lesson: complicated search is not automatically better.

Experiments 9-10 - Early PFSP

These runs were messy because of invalid candidates, timeouts and truncated outputs. They are kept as exploratory evidence only.

Experiment 11 - Clean PFSP

After cleaning the test, independent Best-of-N beat the behavioral quality-diversity method. A scored 531,168.2 vs 531,503.2; A won 55 comparisons, C won 4, and 61 were ties. The reported Wilcoxon p-value was below 0.0001.

Experiment 12 - Targeted refinement

A plan existed to spend the last five calls refining a strong candidate. The final numerical record is incomplete, so this experiment is not used as evidence.

Experiment 13 - Cloud scheduling

Amaura found a cloud scheduling policy with a reported hidden score of 2,099.51 vs 2,171.13 for kube_binpacking, about 3.30% better. But only 1 of 5 search trials found it. Also, Exp14 reports the same 120-workload hidden-set context and the same 2,171.13 reference. Because the raw manifest is not preserved, exact byte identity cannot be hashed; Exp14 is treated as exploratory.

Experiment 14 - More attempts

We tried 15, 30 and 60 independent attempts. In this small sample, more attempts improved the average and reduced spread, but even N=60 had a mean of 2,183.80, still worse than the 2,171.13 baseline.

Experiment 15 - Production-derived data

We moved to Alibaba Cluster Trace data. Static performance mostly held, but the old cloud winner failed on dynamic workloads. The important discovery was the reason: machines need to drain and shut down efficiently when workloads leave. A simple first_fit policy actually beat the old Amaura champion on the dynamic replay: $1,252.29 vs $1,256.26 for kube_binpacking.

Experiment 16 - Dynamic cloud search

Amaura then searched specifically for churn, drainability and active-node behavior. The reported T01_C48 result was $12,509.21 vs $12,562.59, a $53.38 difference. But the later audit found that the dynamic trace context was reused from Exp15, so the result is **contaminated / reused** and cannot count as fresh evidence.

Extra important: first_fit-level behavior explains about $39.70 of the $53.38 gain - roughly **74%**. The active-node barrier adds only $13.68. No-barrier node-hours exactly match first_fit at 3,069.34. T01_C48 vs first_fit was 4 wins / 1 tie / 5 losses, p=.8203.

Lesson: Experiment 16 is still useful, but mainly as a diagnostic. We have not yet earned the claim that Amaura discovered a new dynamic scheduling principle.

7. The Biggest Scientific Lessons

Lesson 1: An evaluator is everything. The system can only optimize what the evaluator measures.

Lesson 2: A good prompt can already be very powerful. We still need a direct experiment comparing expert prompting with Amaura.

Lesson 3: Simple independent Best-of-N currently has better support than the more complicated evolutionary/QD variants we tested.

Lesson 4: A hidden test only counts as hidden if it was really untouched. Reusing a test set after learning from it burns the holdout.

Lesson 5: A failure can be scientifically valuable. Experiment 15 told us exactly what Experiment 16 needed to investigate, and also created the contamination risk that now has to be acknowledged.

Lesson 6: New-looking code is not automatically a new scientific invention. A model can combine old ideas into a useful algorithm without inventing a fundamentally new primitive.

8. Prompting vs Amaura

Your objection was correct: a skilled person may simply write a very good prompt and get a strong answer from the same model.

Method	What it tells us
Expert prompt, one answer	How good is a strong one-shot answer?
Expert prompt, many answers	How much does repetition alone help?
Amaura, many answers + evaluator	Does objective testing and selection add value?
Amaura + expert prompt	Does search still add value after prompt quality is controlled?

This is one of our most important next experiments. We should not assume the answer. We should measure it.

9. Why the Bigger Hypothesis Is Still Open

The broader vision is about problems where “better” is not a single obvious number. For a business, design, science or physical experiment, humans may need to define several goals and constraints. That means the evaluator becomes the bridge between human intention and machine search.

So the big question is not simply “Can AI generate ideas?” AI already does that. The big question is: **Can we build trustworthy evaluators and search systems that repeatedly turn AI possibilities into verified improvements across broader problem classes?**

10. Stage 2 - What Must Happen

Step	Simple rule
1. Fresh benchmark	Use Alibaba windows never touched by Stage 1, or use a completely different trace family.
2. Hash before search	Hash every raw trace and the canonical manifest before any AI candidate is generated.
3. Store hash outside workspace	Keep the reservation record separate so it cannot be silently changed.
4. Freeze evaluator	Freeze evaluator code/version before search begins.
5. Freeze baseline	Freeze baseline code/version and runtime environment.
6. Independent review	Have a human review the evaluator and run adversarial tests before search.
7. Prompting test	Directly compare expert prompting vs Amaura under matched budget.
8. Final holdout	Do not touch the reserved final traces until the winner is frozen.

Current status: We have not actually reserved new raw traces in this workspace because the raw benchmark manifests are not available here. The rule is written down so the next Stage 2 experiment starts correctly. For Stage 2,

treat hidden_dyn_00 through hidden_dyn_09 as burned. The ordered reported-label list has SHA-256 a5eb09e515ade3fcd374ad62bde9e31e3cbe389b8378ed3eaa162bd30fda3baf; this is not a raw data hash.

11. AI-Assisted Research Disclosure

The experiment pipeline, evaluators, runner code and analysis/reporting were developed with substantial AI assistance. The current archive does not support a reliable human-vs-AI percentage split, so no percentage is claimed.

We also do not have a formal independent line-by-line evaluator review record in the accessible archive. That is now a Stage 2 requirement. Experiment 1 showed exactly why this matters: evaluator/pipeline mistakes can create false evidence.

12. Related Work in Simple Words

We are not the first people to build “AI makes code + computer tests code + search keeps the best.” FunSearch already demonstrated an LLM plus automated evaluation for mathematical and combinatorial discovery. AlphaEvolve expanded the idea into a broader evolutionary coding agent for algorithm design and optimization. OpenEvolve is an open-source implementation inspired by AlphaEvolve.

What this means for Amaura: the basic loop is prior art. Our research must therefore focus on what Amaura adds beyond that loop - for example evaluator quality, independent-search reliability, cost efficiency, reproducibility, cross-problem transfer and customer workflow value.

13. Revision Notes from v1.0

The final Stage 1 paper is not just a prettier copy of v1.0. It changes the interpretation in several places: Experiment 16 is no longer treated as fresh validation; the first_fit contribution is quantified; Exp13/14 hidden-set reuse is disclosed; the Experiment 14 Best-column error is corrected; AI assistance in pipeline/evaluator development is disclosed; the raw trace manifests are acknowledged as unavailable; a related-work boundary is added; and a fresh-holdout reservation procedure is defined for Stage 2.

14. Cheat Sheet

Word	Think of it as
AI model	The cook
Candidate	One recipe
Evaluator	Taste test
Search	Trying many recipes
Best-of-N	Try N recipes and keep the best measured one
Hidden test	Blind taste test
Ablation	Remove one part and see what changes
Generalization	Does the recipe still work in a new kitchen?
Provenance	Receipt showing what actually happened
Contamination	Using the blind test after already learning from it

One sentence to remember: Amaura is not trying to make AI magically smarter. It is trying to make AI **testable** - generate possibilities, run experiments, measure reality, and keep what survives.

15. Source Notes

This guide is based on the Amaura research record, experiment transcripts and the corrected Stage 1 official paper. It intentionally does not invent the missing Experiments 3-5 or the incomplete Experiment 12 result.

Web-verified related-work sources: Google DeepMind, “FunSearch: Making new discoveries in mathematical sciences using Large Language Models” (2023); Google DeepMind, “AlphaEvolve: A Gemini-powered coding agent for designing advanced algorithms” (2025); OpenEvolve open-source repository. These sources establish prior art for the general LLM + evaluator + search pattern.